

Group Number: 7

Binny Bajaj, Michael Miranda, Vincent Hayes Bursey

Project Topic - Transportation and Urban Mobility Analytics

1. Project Status Update

Business Problem: Traditional taxi operations often judge performance using gross revenue or total trip volume. However, these high-level metrics mask operational inefficiencies, such as long driver wait times, uncompensated vehicle repositioning, and unbalanced supply-demand zones. The objective of this project is to analyze spatial-temporal trip patterns to optimize vehicle positioning, maximize passenger demand capture, and evaluate revenue efficiency through metrics like revenue per mile and revenue per minute.

Selected Dataset:

- NYC Yellow Taxi Trip Data (Q1 2015 & Q1 2016): A transactional dataset spanning January–March for both 2015 and 2016, totaling 73,010,683 raw records. The dataset captures meter engage/disengage timestamps, passenger counts, trip distances, coordinate/zone location identifiers, and detailed financial breakdowns.
- NYC Taxi Zone Lookup Table: Official reference dataset uploaded to translate location identifiers into human-readable zone and borough names.

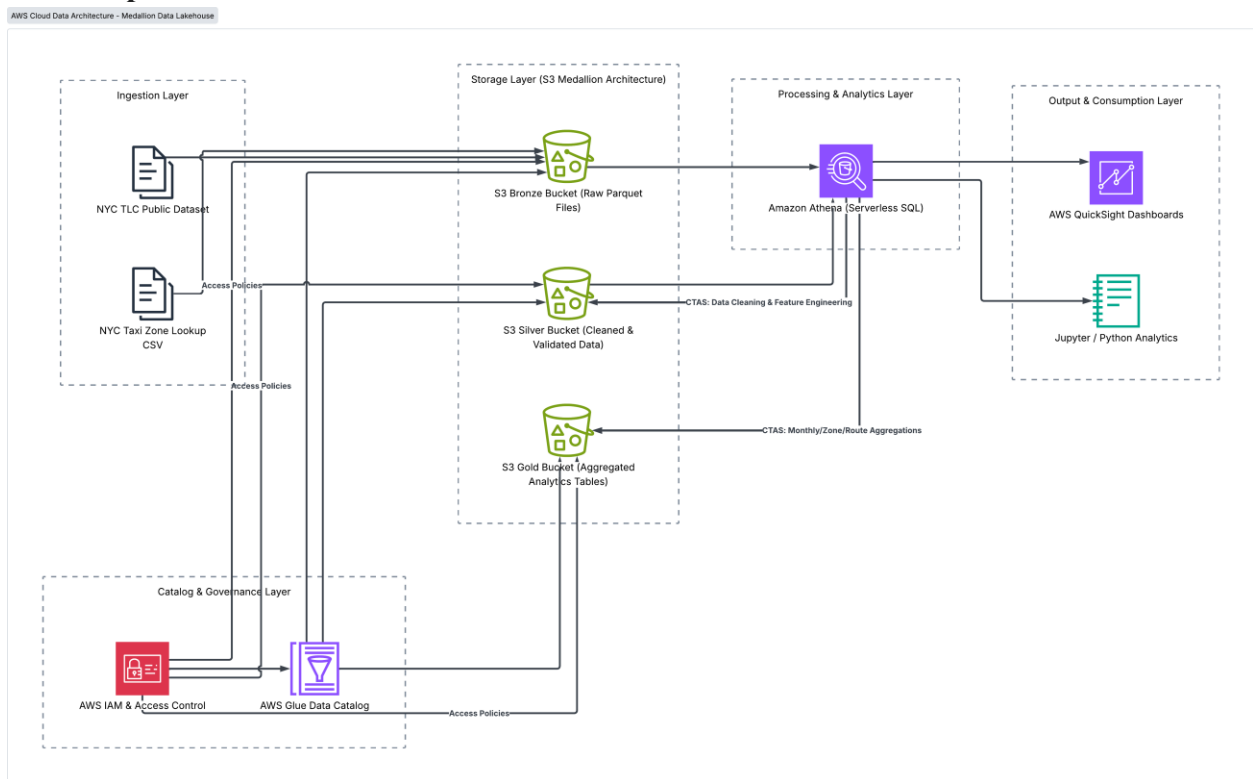
Summary of Changes from Initial Project Proposal:

Since the initial proposal, several key technical and architectural refinements were made based on real-world execution and cost-performance profiling:

- Expanded Time Scope (2015 vs. 2016): Expanded the dataset beyond Q1 2016 to include Q1 2015, enabling direct Year-over-Year (YoY) comparative analytics across 73+ million total trip records.
- Direct Ingestion of Parquet Formats: Rather than landing raw uncompressed CSV files into S3 and relying on AWS Glue ETL jobs for initial conversion, Parquet files were directly ingested into the Bronze layer, significantly reducing landing storage size and initial scan costs.
- Simplification to an Athena Lakehouse Architecture:
 - Initial Plan: Proposed loading data into an Amazon Redshift Serverless data warehouse using a star schema.
 - Current Implementation: Eliminated Redshift in favor of an Athena-only Data Lakehouse model. Data cleaning, schema validation, feature engineering, and dimensional aggregations were executed directly over S3 Parquet tables using Athena Create Table As Select (CTAS) queries. This removed cluster provisioning overhead while achieving sub-second query performance at a fraction of the cost.

- Formalized Medallion Architecture: Established explicitly S3 storage prefixes matching industry standards
 - Bronze: Raw historical trip files & lookup tables
 - Silver: Cleaned dataset (72,265,156 records retained) with calculated fields
 - Gold: Pre-aggregated analytical tables
- Primary Objective: Deliver an interactive, data-driven BI dashboard (via AWS QuickSight / Python Plotly) backed by optimized S3/Athena Gold tables to identify high-efficiency routes, peak spatial-temporal demand windows, and zone supply-demand imbalances (Net Flow = Pickups \$-\$ Drop-offs).
- Actionable Business Outcome: Provide operations managers and taxi drivers with targeted, empirical recommendations on where and when to position vehicles to minimize idle time, reduce empty repositioning miles, and maximize hourly earning capacity.

2. Updated Architecture



Main Changes:

- Removed Redshift. We successfully executed the Bronze --> Silver --> Gold transformation, profiling, and aggregation pipeline entirely inside Amazon Athena via SQL

- Standardized source data to Parquet for the Ingestion Layer. Updated the Raw CSV Files to reflect that the source files landed directly as Yellow Taxi Parquet Files for Jan – Mar 2015 and 2016
- Included the reference dataset (NYC Taxi Zone Lookup). We explicitly uploaded and cataloged the zone lookup table so Athena could join location IDs with readable zone and borough names for the Gold tables (gold_zone_performance and gold_route_performance)

AWS Services & Architecture Implementation

- 1) Storage Layer: Amazon Simple Storage Service (Amazon S3)
 - a. Role: Serves as the central Data Lake storage foundation structured around the Medallion Architecture.
 - b. Key Benefits: High durability, low-cost object storage, and native partition organization (Year / Month) to minimize query scan volume.
- 2) Catalog & Governance Layer: AWS Glue Data Catalog
 - a. Role: Acts as the centralized metadata repository and schema registry across the data lake lakehouse.
 - b. Implementation: Stores table definitions, column types, and partition metadata for all three Medallion tiers and reference lookup tables, allowing Athena to query S3 objects as structured SQL tables without data movement.
 - c. Key Benefits: Decouples data storage from metadata management and enables seamless schema discovery via Glue Crawlers / DDL statements.
- 3) Compute & Analytics Layer: Amazon Athena
 - a. Role: Provides a serverless, interactive query engine executing ANSI SQL directly over data in S3.
 - b. Implementation: Used for all stages of data transformation and querying
 - i. Profiling & Data Quality: Running checks to identify negative fares, zero distances, and invalid records.
 - ii. ETL & Feature Engineering: Executing CTAS (Create Table As Select) statements to transform raw data into Snappy-compressed Silver Parquet tables with calculated metrics (revenue_per_mile, revenue_per_minute).
 - iii. Analytical Aggregations: Building Gold layer tables (gold_monthly_performance, gold_zone_performance, gold_route_performance) and driving business query executions.
 - c. Key Benefits: Fully serverless (no server or cluster provisioning required) with a pay-per-query model based on data scanned.

3. Partial Implementation Evidence

- a. Provide screenshots, logs, SQL outputs, notebook outputs, or other evidence of progress.

- b. Examples may include data uploaded to cloud storage, cataloged tables, Athena/SQL queries, transformed files, warehouse tables, notebook analysis, visualizations, or data quality checks.
- c. The goal is to show that your group has started building or testing key parts of the pipeline.

Figure 1. NYC Yellow Taxi Parquet files stored in the S3 raw layer and partitioned by year and month.

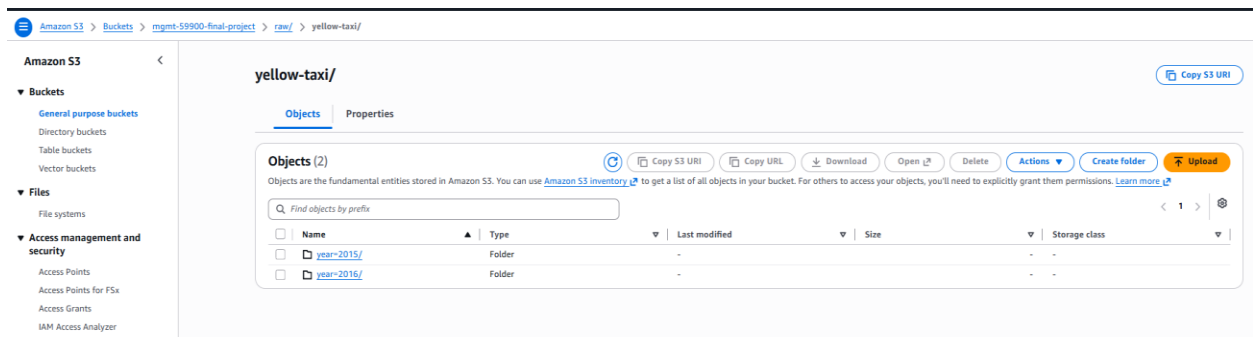


Figure 2. Amazon Athena successfully queried the cataloged `yellow_taxi` table across six year/month partitions covering January–March 2015 and January–March 2016.

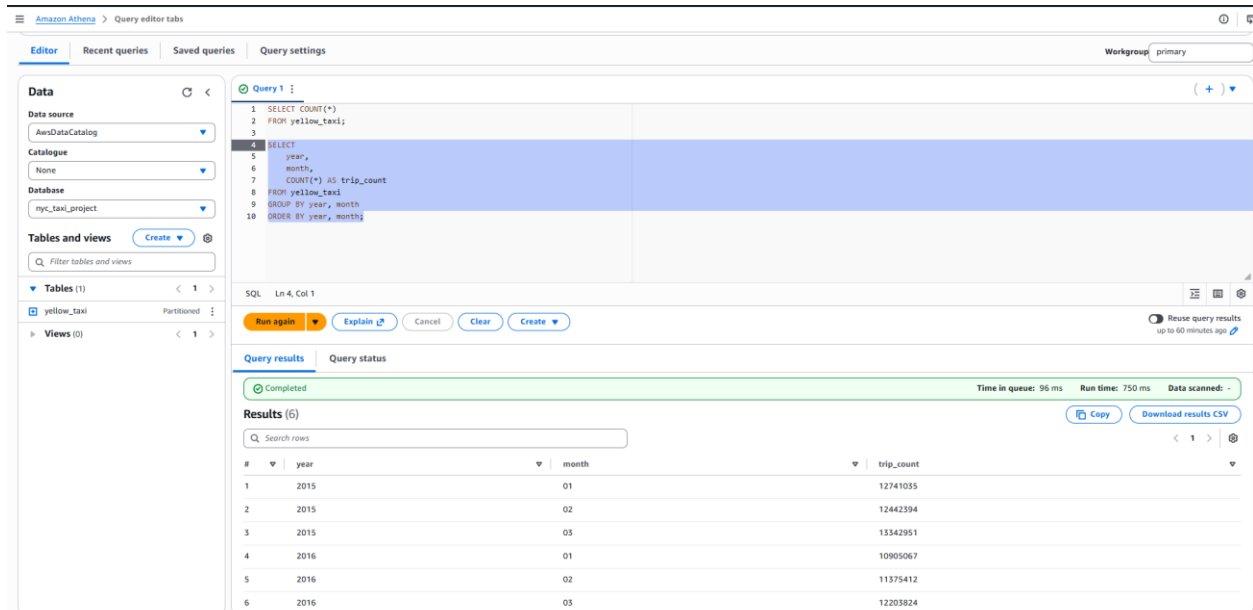


Figure 3. Raw data-quality result showing the invalid-value counts

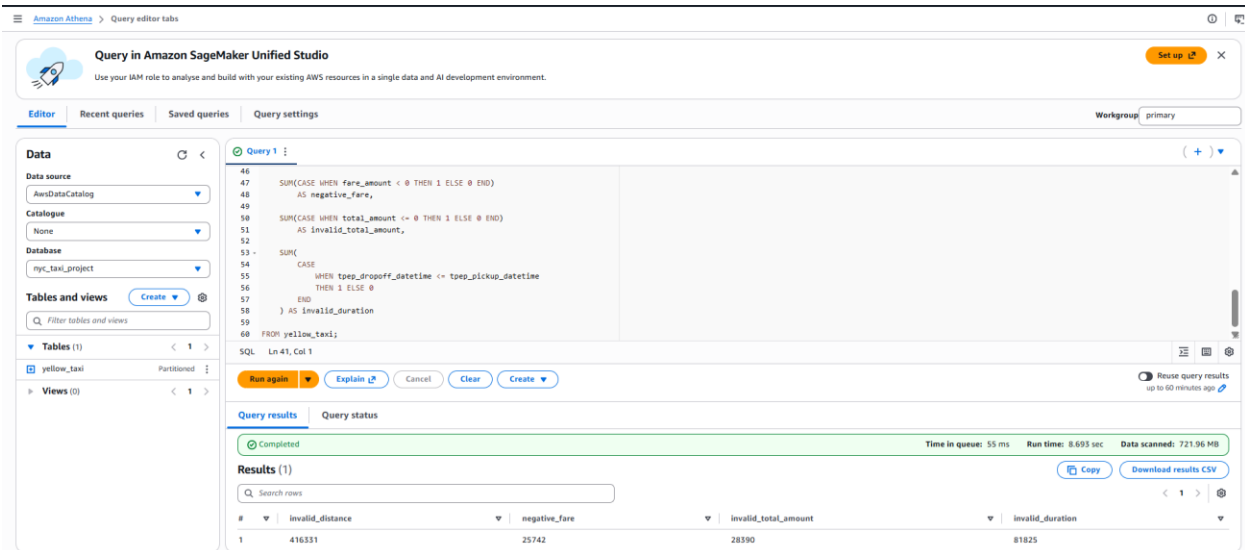


Figure 4. Validation of the Silver dataset confirmed that records violating the defined distance, fare, total amount, and duration rules had been removed.

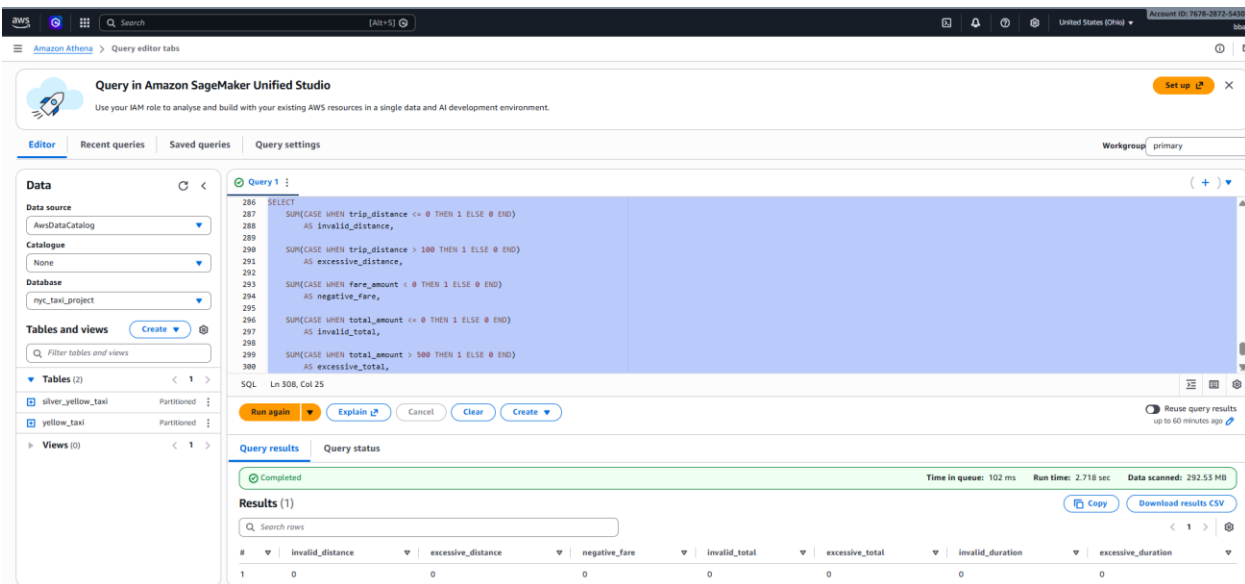


Figure 5. Top 10 pickup zones queried with gold_zone_performance table:

The screenshot displays the Amazon Athena Query Editor interface. On the left, the 'Catalogue' is set to 'None' and the 'Database' is 'nyc_taxi_project'. The 'Tables and views' section lists several tables: 'gold_monthly_performance', 'gold_zone_performance', 'silver_yellow_taxi', 'taxi_zone_lookup', and 'yellow_taxi'. The main query editor shows a SQL query to find the top 10 pickup zones by revenue. The query is as follows:

```

484
485 ## top 10 pickup zones
486 SELECT
487     pickup_borough,
488     pickup_zone,
489     SUM(total_trips) AS trips,
490     SUM(total_revenue) AS revenue
491 FROM gold_zone_performance
492 GROUP BY
493     pickup_borough,
494     pickup_zone
495 ORDER BY trips DESC
496 LIMIT 10;

```

The query status is 'Completed'. The results are displayed in a table with 10 rows and 5 columns: '#', 'pickup_borough', 'pickup_zone', 'trips', and 'revenue'.

#	pickup_borough	pickup_zone	trips	revenue
1	"Manhattan"	"Upper East Side South"	2645163	3.132433382002095E7
2	"Manhattan"	"Midtown Center"	2582324	3.671924221002538E7
3	"Manhattan"	"Upper East Side North"	2462875	3.050531715001807E7
4	"Manhattan"	"Union Sq"	2420960	3.249260812001793E7
5	"Manhattan"	"East Village"	2407921	3.429393905002075E7
6	"Manhattan"	"Midtown East"	2406420	3.445747180002031E7
7	"Manhattan"	"Times Sq/Theatre District"	2395616	3.639188484002452E7
8	"Manhattan"	"Murray Hill"	2349788	3.235145277001740E7
9	"Manhattan"	"Penn Station/Madison Sq West"	2322422	3.222742273001856E7
10	"Manhattan"	"Clinton East"	2252372	3.1267172010017324E7

4. Data Model and Query Plan

- Describe your current data model or planned structure.
 - Our current model uses a bronze, silver, and gold data model in amazon S3.
 - Bronze folder containing the original Taxi Parquet files, silver stored in the cleaned trip-level data, and gold stores aggregated business-ready tables.
 - AWS Glue catalogs the data, whereas Athena is used for profiling, transformation, and aggregation.
- Identify important tables, files, entities, keys, partitions, or fact/dimension tables where applicable.
 - Yellow_taxi is the raw partitioned file, while silver_yellow_taxi is the clean trip-level table. The data is partitioned by year and month.
 - Pickup_location_id and dropoff_location_id is joined with taxizone_lookup to identify related taxi zones and boroughs.
 - Also, gold tables include gold_monthly_performance, gold_zone_performance, and gold_route_performance.
- Explain how the data will support your final analysis, dashboard, or model output.
 - These gold tables will be used as a source for final dashboard and support the analysis of trip demand, revenue efficiency, high-performing zones, and routes.
 - The Redshift Serverless star schema is planned for final implementation, where aggregated trip-performance data will be organized into a fact table with supporting dimensions such as date and taxi zone, along with additional dimensions such as vendor and rate code, can also be incorporated if they provide useful analytical value.

5. Data Quality, Governance, and Cleanup

1) Potential Data Quality & Anomaly Issues

Even though our Silver cleaning step successfully filtered standard edge cases, retaining ~99% of data, big data pipelines run into subtle domain anomalies:

- Geospatial Outliers: Coordinates defaulting to (0.0, 0.0) or points falling outside the greater NYC tri-state boundary (e.g., GPS drift putting trip coordinates in the Atlantic Ocean).
- Logical & Chronological Inconsistencies:
 - Drop-off times occurring before pickup times.
 - Zero-mile trip distances with high fare amounts (e.g., meter turned on/off without driving) or long distances with \$0.00 fares.
- Financial Anomalies: Extreme tips (e.g., fat-finger entry like a \$5,000 tip on a \$15 fare), negative toll fees, or missing total_amount components.
- Missing Location Map Join Keys: Unmatched LocationID values in the transaction records that fail to join against the NYC Taxi Zone Lookup table, resulting in NULL or "N/A" zones.

2) Cleanup & Governance Documentation Steps

To ensure our pipeline remains compliant, repeatable, and easily auditable by stakeholders:

A. Data Quality & Profiling Log

- Document a Data Validation Matrix in our final report to show the rule logic and row impacts

Category	Rule Condition	Purpose/Impact	Action
Duration	trip_duration BETWEEN 1 AND 720	Drops unrealistic trip times <1min or >12 hours	Filtered in Silver
Spatial	pickup_latitude BETWEEN 40.4 AND 40.9	Excludes GPS points outside NYC bounds	Filtered in Silver
Financial	fare_amount > 0 AND total_amount > 0	Prevents negative or fraudulent accounting records	Filtered in Silver
Joins	COALESCE(zone_name, 'Unknown')	Handles unmapped lookup IDs without dropping data	Transformed in Gold

B. AWS Glue Data Catalog Governance

- **Schema Evolution & Enforcement:** Use explicit column data types in the Data Catalog rather than relying solely on automated inference to prevent accidental type casting (e.g., ensuring passenger_count stays INT rather than BIGINT or DOUBLE).
- **Metadata Lineage:** Tag catalog tables with ownership, environment (dev/prod), and tier (bronze, silver, gold) tags to maintain clear data lineage.

3) Cloud Cost Management & Resource Cleanup:

Because cloud data projects can quietly accrue costs from lingering storage and catalog resources, the following tear-down process will be implemented:

A. Active Cost Control (Now)

- **Athena Query Partition Safeguards:** Always enforce WHERE year = '2016' AND month = '01' filters when running exploratory queries to limit data scanned per query.
- **AWS Budget Alerts:** Set an active AWS Cost Budget with email/SMS alerts triggered at \$5.00 total spend.

B. Decommissioning & Infrastructure Clean-Up

When the project is complete, we will execute the following to prevent dependency errors:

1. Storage Cleanup (Amazon S3)

- a. Empty all objects inside mgmt-59900-final-project (including hidden versions/multipart uploads), then delete the bucket containers.

2. Catalog Cleanup (AWS Glue)

- a. Delete the custom database and registered tables/crawlers in the AWS Glue Data Catalog to prevent automated background crawlers from attempting scheduled runs.

3. Query History & Workgroups (Amazon Athena)

- a. Clear Athena query result files stored under “athena-results” before deleting the bucket

4. IAM Role & Permission Cleanup

- a. Delete custom IAM roles/policies created specifically for Athena S3 full-access or Glue execution.

5. Billing Audit

- a. Review AWS Cost Explorer 24–48 hours post-cleanup to confirm daily active spend drops to strictly \$0.00.

6. Updated Risks and Next Steps

Potential Risks Summary Table:

Risk Category	High Level Risk	Mitigation Strategy
Performance	Small file overhead in S3 Parquet	Compacting output files to 128MB–512MB in CTAS
Data Integrity	Unregistered S3 partitions	Using MSCK REPAIR TABLE or Partition Projection
Spatial Skew	GPS drift & (0,0) coordinate anomalies	Applying explicit NYC bounding-box filters in SQL
Financial/Cost	High Athena scan costs from BI queries	Connecting QuickSight solely to Gold aggregated tables
Schema Evolution	Data type shifts between 2015 and 2016	Hardcoding column type casting in DDL statements
Metrics Distortion	Extreme outliers (zero miles, flat tips)	Applying statistical range filters for speed/duration/fare

Top 2 Potential Risks:

1) Small File in S3 & Athena May Impact Performance + Bottlenecks

- Risk: Running multiple CREATE TABLE AS SELECT (CTAS) statements without controlling writer parallelism can output hundreds of tiny Parquet files (<10 MB) into your S3 Silver or Gold folders. Reading thousands of small files causes severe metadata overhead in Athena, resulting in slow query performance and unnecessary compute costs.
- Mitigation Strategy:
 - Target an optimal file size of 128 MB to 512 MB per Parquet object.
 - In PySpark or Athena CTAS jobs, use bucket or file-compaction options (e.g., controlling bucket_count or running periodic compaction scripts) so that each partition folder contains a small number of well-sized files.

2) Partition Drift & Missing Partition Metadata

- Risk: When new data is written directly to S3 under hive-style paths (e.g., s3://.../year=2016/month=03/), Athena will not automatically query it until the Glue Data Catalog is updated. If team members add raw files or re-run Silver CTAS jobs without updating the metadata, queries will silently return incomplete data without throwing errors.
- Mitigation Strategy:
 - Automate catalog updates by executing MSCK REPAIR TABLE <table_name> after every data write.

- Alternatively, implement Athena Partition Projection in your table properties, allowing Athena to dynamically compute partition locations based on template rules rather than querying the Glue metastore.

Work To Be Done:

- Finalize the technical checkpoint write-up and select the strongest screenshots showing S3 ingestion, Athena profiling, Silver validation, and Gold analysis.
- Update the architecture diagram to reflect the current Bronze/Silver/Gold implementation.
- Decide whether to implement the proposed Redshift Serverless star schema for the final project because everything can be done in Athena itself.
- Potentially create a pickup/drop-off zone-balance analysis to support the vehicle-repositioning part of our business problem.
- Build the final QuickSight dashboard or notebook visualizations using the Gold tables.
- Compare 2015 versus 2016 performance and identify the key business insights/recommendations.
- Complete cost/cleanup documentation and remove cloud resources after the project.